

Implementation Plan — AI Nutrition Assistant

Phased build order derived from [problemStatement.md](#) and [rag-architecture.md](#). Each phase ends in a runnable, verifiable state; later phases assume earlier ones are done. "Exit criteria" are what to check before moving on.

Part	Milestone	Phases	Status
A	Milestone 1 — prototype without retrieval	0–11	[DONE] Complete and deployed
B	Milestone 2 — dietary guidance RAG	12–22	[TODO] Not started

Part A — Milestone 1 (Complete)

Condensed to what actually shipped, since this is now the baseline Milestone 2 builds on rather than work to be done. Where the build diverged from the original plan, the divergence is recorded — those are the useful parts.

Phase	Deliverable	Outcome
0	Scaffolded Next.js 14 App Router + TypeScript + Tailwind repo	[DONE]
1	<code>prisma/schema.prisma</code> — Conversation, Message, Claim, EvalRun, FailureLogEntry; <code>lib/db.ts</code> singleton	[DONE] Postgres for local dev too , not the planned SQLite-dev/Postgres-prod split — Vercel's filesystem is ephemeral, and one database avoids a dev/prod schema divergence
2	<code>lib/schema.ts</code> , <code>lib/systemPrompt.ts</code>	[DONE] Zod as single source of truth, converted to the model's JSON-schema format <i>and</i> reused for validation
3	<code>lib/groq.ts</code> — model call wrapper	[DONE] Structured Outputs (<code>response_format: json_schema, strict: true</code>), not the planned forced tool-calling . <code>gpt-oss</code> reasoning models intermittently emit chain-of-thought under forced <code>tool_choice</code> , which Groq's tool-call parser rejects with a 400
4	<code>lib/scopeGuard.ts</code> + <code>scripts/test-scope-guard.ts</code>	[DONE] Regex-based, not a classifier. Unit-tested against 12 restricted phrasings, the 10 eval questions, and 5 benign counter-examples
5	<code>app/api/chat/route.ts</code> (POST + DELETE)	[DONE] 12-step pipeline, rag-architecture.md §11
6	<code>ChatWindow</code> , <code>MessageBubble</code> , <code>ChatInput</code> , <code>SourcesPanel</code>	[DONE] Sources panel built with its populated branch written but unreachable — by design
7	<code>data/eval-questions.json</code> + <code>scripts/evaluate.ts</code>	[DONE] 10 questions × 3 attempts, fresh conversation each
8	End-to-end scope-abuse suite	[DONE] 8 cases: <code>numeric_target</code> / <code>medical_advice</code> × direct / rephrased / indirect / post-unrelated
9	Prompt iteration loop	[DONE] See prompt-iteration-log.md
10	Deployment	[DONE] Both Vercel and Railway , sharing one Railway Postgres
11	Milestone 2 readiness check	(!) Passed at the time, but two findings have since superseded it — see below

Recorded Milestone 1 baseline

[Docs/failure-log.md](#), prompt version `eabb1ca8b1`: **4 × numeric_drift** — protein requirements, egg storage, cooked-rice storage, blanching time. This is the number Milestone 2 is measured against.

Two corrections to the Phase 11 readiness conclusion

Phase 11 concluded that Milestone 2 could begin "without touching the schema, API contract, or UI components." Design work on the retrieval layer has shown **two parts of that are wrong**, and Part B is planned accordingly:

1. **The schema does have to change.** `lib/schema.ts` plans to widen `source` to `z.string().url().nullable()`. A bare URL cannot carry document name, publisher and year, all of which Milestone 2 requires on every claim. `source` becomes a structured object ([rag-architecture.md §20.1](#)). The *field name and response envelope* are still unchanged, which is what the problem statement actually froze.
2. **SourcesPanel does need structural work.** Its populated branch renders `claim.source` as both the `href` and the visible link text — correct only for a bare URL. It must be rewritten to render document, publisher, year, section, link and the cited chunk text ([rag-architecture.md §22](#)).

The API contract conclusion holds: `{ conversationId, answer, claims[] }` keeps its shape, with `retrieval` added alongside.

Part B — Milestone 2: Dietary Guidance RAG

Build order rationale. The corpus comes first, because every later phase depends on what is actually in it — and corpus research has already shown that what is in it differs from what was assumed. Retrieval is built and measured *before* the answer layer, so a retrieval problem is never debugged as a generation problem.



Phase 12 — Corpus Selection and Manifest

Goal: a fixed, verified set of 5–7 documents, with provenance recorded, before a line of pipeline code is written.

Tasks

- Choose the final 5–7 from the 39 checked URLs in [problemStatement.md](#). Starting shortlist is the recommended seven: WHO Healthy Diet fact sheet, EFSA DRV Summary, DGA 2025–2030, Eatwell Guide, WHO Five Keys, FSA chill/freeze/defrost, WHO sodium guideline.
- Create `corpus/manifest.json` with, per document: `name`, `publisher`, `year`, `url` (reader-facing), `fileUrl` (fetchable — these differ for EFSA), `retrievedAt`, `edition`, `acquisition`, `sourceFile`, `licenseNote`, `expectTitleContains`, `expectYearIn`.
- **Acquire the four bot-blocked documents by hand** and commit them to `corpus/raw/`: FoodSafety.gov cold-storage charts, USDA FSIS temperature chart, and any others returning 403. Record `acquisition: "manual"`.
- Record the **corpus boundary** — what the corpus does *not* cover. Note that children's requirements will **not** work as the boundary: WHO's sodium guideline covers ages 2–15 and the US guidelines cover birth onward. Pick clinical/therapeutic diets, pregnancy-specific requirements, or an unaddressed food category instead.
- Record which topic is the **deliberate cross-document overlap**. The verified candidate is leftover storage: FSA says *within 48 hours*; FoodSafety.gov says *3–4 days*.
- Confirm the US "3–4 days" figure against the hand-acquired document. It currently comes from a search-engine rendering, not the source.

Exit criteria

- `corpus/manifest.json` lists 5–7 documents; every `sourceFile` exists in `corpus/raw/` and is committed.
- Every `year` was read from inside the document, not from the page that linked it.
- The corpus boundary and the overlap topic are written down.

Do not skip the edition check. Two of eleven candidates returned HTTP 200 while being the wrong edition: the widely-cited *DGA 2020–2025* is superseded by the 2025–2030 edition (Jan 2026), and the only downloadable ICMR-NIN PDF is the **2011** manual, not the 2024 revision. Both would produce a *fabricated citation behind a working link* — the hardest failure to notice.

Phase 13 — Data Model Migration (Additive Only)

Goal: the corpus and citation tables, in a migration safe to run while one deploy target still runs old code.

Tasks

- Enable the `pgvector` extension on the Railway Postgres.
- Add `Document`, `Chunk` (with `embedding vector(1536)`, `restricted`, `configHash`), and `RetrievalRecord` per [rag-architecture.md §21.2](#).
- Add `Claim.chunkId` as a **nullable** FK to `Chunk`.
- **Leave `Claim.source` in place.** It is dropped in a later release, not this one.
- Add `lib/retrievalConfig.ts` — one frozen object (chunk target, cap, overlap, embedding model, index type, `k`, sufficiency thresholds) plus its `sha256` prefix.

Exit criteria

- Migration applies cleanly; `npx prisma studio` shows the new tables empty.
- **The migration is additive only** — nothing dropped, nothing made non-nullable, no column removed. Verify by reading the generated SQL, not by assuming.
- Milestone 1's existing `/api/chat` still works unchanged against the migrated database.

Phase 14 — Ingestion Pipeline

Goal: `scripts/ingest.ts` (`npm run ingest`) — reproducible, idempotent, offline-capable.

Tasks — the nine steps from [rag-architecture.md §10](#):

1. **Acquire** — fetch `fileUrl`, or read the committed file for `acquisition: "manual"`.
2. **Checksum** — `sha256` into `Document.checksum`; warn loudly if it changed since the last run.
3. **Verify** — read title and year from inside the document; assert `expectTitleContains` / `expectYearIn`; **abort the run on mismatch**.
4. **Extract** — PDF → pages, headings, text.
5. **Quality gate** — **per page, not per document**. Drop pages below a words-per-page threshold; keep the document.
6. **Chunk** — heading-aware, 500-token target, 900 hard cap, 80-token overlap, never split a detected table.
7. **Policy scan** — set `Chunk.restricted = true` on passages carrying calorie or per-kg-body-weight targets.
8. **Embed** — batch to the embedding provider.
9. **Upsert** — `Document` + `Chunk`, keyed by `(name, year, edition)` and `configHash`.

Exit criteria

- Full ingest produces roughly **300–400 chunks** across the corpus.
- Every chunk has a non-null `section`, a `documentId`, a `tokenCount` and an embedding.
- Re-running with an unchanged manifest changes nothing (idempotent).

- A deliberately corrupted `expectYearIn` **aborts the run** — test this explicitly.
- Chunk counts per document are reported and roughly match the estimates below.

Measured inputs this phase must handle (full text extraction, 2026-10-02):

Document	Pages	Words	Words/page	Est. chunks
DGA 2025–2030	10	2,704	270	~7
Eatwell Guide	12	5,166	430	~14
safe food (Ireland)	14	6,085	434	~16
FSANZ	23	6,930	301	~18
Brazil guidelines	152	30,459	200	~81
ICMR-NIN (2011)	139	33,106	238	~88
EFSA DRV Summary	92	48,717	529	~130

Two extraction facts to build against. (1) The Eatwell Guide's page 1 is artwork and extracts as a word-salad of food names; pages 2–12 are clean prose at 430 words/page — which is why the quality gate is **per page**. (2) EFSA is **18× the DGA by word count**, so a single global `k` will let it crowd out current US guidance entirely.

Phase 15 — Retrieval Layer

Goal: `lib/retrieval.ts` and `lib/embeddings.ts` — working vector search in both modes, testable without the API route.

Tasks

- `lib/embeddings.ts`: provider wrapper, batch (ingestion) and single (query), with query caching by normalised hash.
- `lib/retrieval.ts`: `search({ queryVector, k, documentId? })`.
- **All-documents mode** — cosine similarity across `Chunk`, top `k`.
- **Filtered mode** — the `documentId` filter goes in the **SQL WHERE clause**, never as a post-filter on an all-documents result.
- Both modes `JOIN Document` so provenance returns with the text.
- Bind the query vector as a **parameter** in any `$queryRaw`, never interpolated.
- **No fallback on embedding failure** — the request fails rather than answering ungrounded.

Exit criteria

- A throwaway script retrieves sensible chunks for 5 hand-picked questions, in both modes.
- Filtered mode returns `k` chunks from the named document **even when that document is absent from the global top-`k`** — the specific case post-filtering gets wrong.
- Every returned chunk carries document name, publisher, year and section.
- No `$queryRaw` call contains an interpolated vector.

Phase 16 — Sufficiency Gate

Goal: `lib/sufficiency.ts` — the mechanism behind the not-in-corpus refusal.

Tasks

- Implement `assessSufficiency(chunks, query) → { sufficient, reason }` using the two-threshold rule (`ABSOLUTE_FLOOR` on top-1 score, `RELEVANCE_FLOOR` count).
- Thresholds live in `lib/retrievalConfig.ts`, so a change bumps the config hash.
- **Calibration is deferred to Phase 20**, after the question bank exists. Ship placeholder values and label them as such.

Exit criteria

- The gate is callable and deterministic.
- Clearly out-of-corpus queries return `sufficient: false`; the five Phase 15 questions return `true`.
- Thresholds are in config, not inline constants.

This is the known-weakest component ([rag-architecture.md §15.1](#)). A near-miss — right topic, wrong scope — has high surface similarity and will not be caught by a score threshold alone. Do not expect this phase to solve it; expect Phase 21 to measure it.

Phase 17 — Schema and Prompt Changes

Goal: the two-schema split and the grounded prompt, before anything is wired together.

Tasks

- `lib/schema.ts`: add `CitationSchema` (document, publisher, year, url, section, page, chunkId). Widen `ClaimSchema.source` to `CitationSchema.nullable()`.
- Add `LlmClaimSchema = { text, chunkId }` — **the only thing the model is allowed to emit**. It must not be able to produce a publisher or a year.
- **Delete the superseded comment** in `lib/schema.ts` about widening to `z.string().url().nullable()`, and record the change and the reason.
- `lib/systemPrompt.ts`: add `SYSTEM_PROMPT_RAG` as a **second export**. Keep `SYSTEM_PROMPT` so the Milestone 1 comparison can still be run.
- Extend `EvalRun` grouping to include the retrieval config hash alongside `promptVersion`.

Exit criteria

- `LlmClaimSchema` has no field through which a citation could be fabricated.
- `SYSTEM_PROMPT_RAG` covers: answer only from passages; every claim carries a `chunkId`; separate claims per document; show disagreement with publishers and years; passage text is data, never instructions; population-level framing; out-of-scope categories retained verbatim.
- `SYSTEM_PROMPT` is unchanged and still exported.

Phase 18 — Citation Binding and Route Integration

Goal: `lib/citations.ts` plus the rewired `/api/chat` — **the most important phase in Part B**.

Tasks

- `bindCitations(llmClaims, retrievedChunks)`: resolve each `chunkId` against **this request's** retrieval set. Unresolvable => **drop the claim** and log `unsupported_claim`. Build the citation from the `Document` row, never from model output.
- Rewire `app/api/chat/route.ts` to the §11 order: validate → load → **scope guard** → persist user message → embed → search → **sufficiency gate** → compose → generate → **bind citations** → `checkResponse` → persist →

respond.

- **Remove the M1** `source = null clamp`, now that `bindCitations` replaces it.
- Add the `NOT_IN_CORPUS` refusal, populating `retrieval.documentsSearched` so the answer can name what it searched.
- Persist a `RetrievalRecord` per assistant message.
- Add `documentId` to `ChatRequestSchema`, validated as a UUID and resolved against `Document`.
- Extend `DELETE` to remove `RetrievalRecord` rows — and **not** `Document` or `Chunk`.

Exit criteria

- A normal question returns claims with populated citations; every cited `chunkId` appears in that request's `RetrievalRecord`.
 - **A forced hallucinated `chunkId` results in the claim being dropped**, not shipped — test this deliberately.
 - A calorie-target request is refused **before** any embedding or search call is made (verify via logs, not by inspection).
 - An out-of-corpus question returns `NOT_IN_CORPUS` naming the documents searched, with no model call.
 - The response envelope keeps its Milestone 1 shape: { `conversationId`, `answer`, `claims[]` } plus `retrieval`.
 - Retrieval happens **before** the rate-limiter reservation, so chunk tokens are counted.
-

Phase 19 — Frontend

Goal: show the evidence. No rebuild.

Tasks

- **Rewrite** the `SourcesPanel` populated branch — it currently assumes `source` is a bare URL. Render document, publisher, year, section, link, **and the cited chunk text**.
- Group citations by document when claims cite different publishers, so a disagreement reads as a disagreement.
- Distinguish the two refusals: not-in-corpus shows what was searched; out-of-scope shows the professional-referral message with no retrieval block.
- A document-filter UI control is **optional** and not required for completion.

Exit criteria

- Clicking an assistant message shows real citations with openable links.
 - A cross-document answer visibly groups by publisher.
 - The two refusal types are visually distinguishable.
 - Mobile (below `md`) still works — the panel remains hidden there.
-

Phase 20 — Retrieval Question Bank and Harness

Goal: measure retrieval **separately** from answer quality, and lock the configuration.

Tasks

- Write `data/retrieval-questions.json` — **15+ questions, authored after ingestion**, each with the known correct document and section. Cover every document, and include the cross-document overlap question.
- `scripts/evaluate-retrieval.ts` (`npm run eval:retrieval`) reporting: `recall@k` (overall **and per document**), `document_recall@k`, `false_refusal_rate`, `unsupported_claim_rate`, `citation_binding_failures`.

- **Calibrate the Phase 16 sufficiency thresholds** against this bank: admit every question whose correct chunk exists; reject the adversarial not-covered cases.
- **Lock and record** in `corpus/README.md`: chunk size, overlap, embedding model, index type, `k`, sufficiency thresholds, chunking strategy, and **what the strategy cost**.
- Writes `Docs/retrieval-report.md`.

Exit criteria

- Hit rate reported overall **and per document** — a weak document must be visible, not averaged away.
- Thresholds are calibrated against real data, not guessed.
- `corpus/README.md` contains every value the problem statement requires.
- `k` has been tuned **jointly with chunk size against the token budget**, not for quality alone.

Write the bank against the corpus that exists, not the one that was planned. Two candidates turned out to be the wrong edition and four could not be fetched. A bank written against the intended corpus measures a corpus that does not exist.

Phase 21 — Verification

Goal: the four exercises the problem statement requires. They are not substitutes for one another.

21a — Adversarial suite, reported split by refusal type so a correct refusal for the *wrong reason* is visible:

- **not-covered** → `NOT_IN_CORPUS`, naming what was searched.
- **near-miss** → refusal, not an answer from the wrong scope. Built from the Phase 12 corpus boundary.
- **out-of-scope** → calorie target and medical advice, each direct / rephrased / indirect / raised-again-later → out-of-scope refusal, **not** `NOT_IN_CORPUS`.
- **corpus-restricted** (*new in M2*) → ask for a protein target. Must refuse **even though a citable chunk would answer it**.

21b — Citation spot-check, by hand. Take 10 answers, open each cited chunk, confirm every number and named recommendation is actually there. Report per-claim **and** per-answer accuracy. Cannot be automated against the same embeddings that produced the retrieval.

21c — Milestone 1 regression. Same 10 questions, compared against baseline `eabb1ca8b1`.

21d — Scope guard regression. `npm run test:scope` still passes unchanged.

Exit criteria

- `numeric_drift`: **4** → **0** is the target. A number that must appear in a retrieved chunk cannot drift between runs.
- Every out-of-scope case refuses, including the corpus-restricted one.
- Citation spot-check reported honestly, including partial failures.
- **Regressions reported, not just improvements** — a question M1 answered well and M2 refuses is a real cost of grounding, and some are expected.
- `no_clear_answer` questions reported by what they now do: cited guidance, or not-in-corpus refusal.

The corpus-restricted test is the new failure mode. Milestone 1 could not fail this way — it had no chunk to be tempted by. The risk is broader than first assumed: the scope conflict affects **EFSA** (protein at "0.8 to 1.25 g/kg body weight per day", energy in kcal/day) as well as the DGA, and EFSA is the densest document in the corpus.

Phase 22 — Deployment

Goal: live on both targets, with the migration ordered so neither host breaks.

Tasks — order matters, because the deploy triggers are asymmetric:

1. Apply the **additive-only** migration to the shared Postgres.
2. Run `npm run ingest` to populate the corpus.
3. Deploy **Railway** (push) **and Vercel** (`vercel --prod` — Vercel does *not* auto-deploy on push).
4. Verify **both** URLs independently.
5. In a **later** release only, once both hosts run new code, drop `Claim.source`.

Exit criteria

- Both public URLs answer with real citations.
- Both read the same corpus from the shared database.
- An out-of-scope request is refused on both.
- A not-in-corpus request refuses and names what it searched, on both.
- `Claim.source` still exists — dropping it is a later release.

`git push` updates **Railway but not Vercel**. Verify the Vercel URL specifically; do not infer it from a successful push.

Decisions to Lock Before Phase 20

Each changes measured results, so each must be fixed and recorded before the question bank runs (`rag-architecture.md §31`).

#	Decision	Leaning	Settle in
1	Which 5–7 documents	The recommended seven	Phase 12
2	Chunk target / cap / overlap	500 / 900 / 80 tokens	Phase 14, confirm Phase 20
3	Table handling	Keep whole, allow over-cap, flag	Phase 14
4	Embedding model + dimension	OpenAI <code>text-embedding-3-small</code> , 1536	Phase 15
5	<code>k</code>	5 — budget-bound, see Risks	Phase 20
6	Index type	Exact search, no ANN — already settled by corpus size	Phase 13
7	Sufficiency thresholds	Calibrate against the bank	Phase 20
8	<code>restricted</code> chunk policy	Flag and retrieve, never restate targets	Phase 14
9	Near-miss corpus boundary	Not children	Phase 12
10	Document-filter UI	API + eval only	Phase 19

Risks

Risk	Why it matters	Mitigation	Phase
Token budget throttling	At $k=5 \times 500$ tokens, each request costs ~4,200–5,200 tokens against a 7,000 tok/min margin — about 1–1.5 requests/minute . A full evaluation is 60+ calls, so 40+ minutes of throttling per run	Tune k and chunk size together; trim history on grounded calls; consider a higher Groq tier before Phase 20	20, 21
Near-miss answers from the wrong scope	The quietest way this milestone fails — a confident answer from a nearly-correct section	Section metadata in context, prompt instruction, explicit adversarial test. Expect to measure, not eliminate	16, 21
Scope conflict with the corpus	Both the DGA <i>and</i> EFSA state per-kg-body-weight and calorie figures. A leak now arrives with a real government citation attached	Ingestion-time restricted flag applied corpus-wide, not per document; dedicated test	14, 21
Wrong edition ingested	Already hit 2 of 11 candidates. Produces a fabricated citation behind a working link	expectTitleContains / expectYearIn abort the ingest	12, 14
Short documents crowded out	EFSA is 18× the DGA by word count	Per-document recall reporting first; per-document quotas only if needed	14, 20
Asymmetric deploys against one database	One host runs new code, the other old, against the same Postgres	Additive-only migrations; verify both URLs	13, 22
Embedding provider is a new dependency	Groq has no embeddings endpoint, so this is a new credential and failure domain	Offline corpus embedding; hard-fail on query-embed failure with no ungrounded fallback	15

Phase Summary

Phase	Deliverable	Depends on
0–11	Milestone 1 — complete	—
12	Corpus manifest + acquired files	11
13	Additive migration + pgvector + retrievalConfig	11
14	scripts/ingest.ts , populated corpus	12, 13
15	lib/retrieval.ts , lib/embeddings.ts	14
16	lib/sufficiency.ts	15
17	CitationSchema , LlmClaimSchema , SYSTEM_PROMPT_RAG	13
18	lib/citations.ts + rewired /api/chat	15, 16, 17
19	SourcesPanel rewrite	18
20	Question bank, retrieval harness, locked config	18
21	Adversarial, spot-check, M1 regression	19, 20
22	Deployment to both targets	21